

W5_hw2 Report

[1]How to Model the Module with the Gate-Level Method

1. Objectives:

- (1) Binary Coded Decimal(BCD)의 개념을 이해하고 Binary-to BCD Converter의 작동 algorithm을 파악한다.
- (2) Converter Submodule이 어떻게 구동되는지 이해하고 gate-level modeling 방법을 이용해 submodule의 소스코드를 구현한다.
- (3) (2)에서 구현한 하위 모듈의 소스코드를 바탕으로 structural modeling 방법을 이용해 Binary-to-BCD Converter의 소스코드를 구현한다.

2. Theoretical Approach:

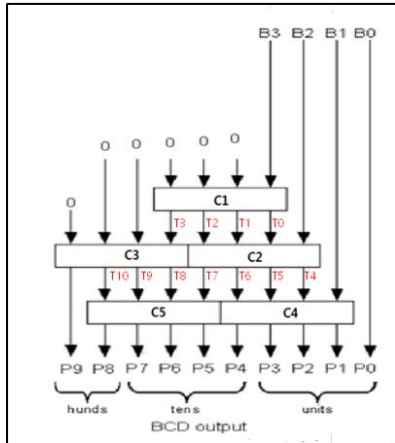
10 진수	2 진수	10의 자리		1의 자리	
		BCD	display	BCD	display
0	0000	0000	0	0000	0
1	0001	0000	0	0001	1
2	0010	0000	0	0010	2
3	0011	0000	0	0011	3
4	0100	0000	0	0100	4
5	0101	0000	0	0101	5
6	0110	0000	0	0110	6
7	0111	0000	0	0111	7
8	1000	0000	0	1000	8
9	1001	0000	0	1001	9
10	1010	0001	1	0000	0
11	1011	0001	1	0001	1
12	1100	0001	1	0010	2
13	1101	0001	1	0011	3
14	1110	0001	1	0100	4
15	1111	0001	1	0101	5

BCD는 decimal number를 binary number로 나타내는 방법의 일종이다. 각 digit을 4-bit로 표현하는데 왼쪽의 표에 나타난 바와 같이 각 4-bit의 범위는 0000 ~ 1001이고 1010 ~ 1111은 사용되지 않는다. 왼쪽의 표는 4-bit의 2진수를 BCD로 나타낸 것으로서 15까지의 input만 나타내었다. Binary number를 BCD로 변환하는데 shift and add-3 algorithm이 사용된다. 왼쪽으로 3번 shift하고 3을 더한 다음 다시 왼쪽으로 1번 shift해서 최종적으로 BCD로 나타내는 알고리즘이다. 알고리즘이 적용되는 단계를 아래의 표로 나타내었다.

연산	10의 자리	1의 자리	Binary
HEX			D
Start			1 1 0 1
1st Left Shift		1	1 0 1
2nd Left Shift		1 1	0 1
3rd Left Shift		1 1 0	1
Add 3		1 0 0 1	1
4th Left Shift	1	0 0 1 1	
BCD	1	3	

A3	A2	A1	A0	S3	S2	S1	S0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

Shift 하고 add 하는 과정이 submodule로 들어가고 그 모듈들이 단계별로 진행되어 최종적으로 10-bit의 BCD가 출력된다. 앞의 2-bit는 백의 자리로 의미는 없다. 왼쪽의 Truth Table을 바탕으로 converter submodules가 진행된다. 이를



바탕으로 Binary-to-BCD converter를 structural modeling하여 diagram으로 도식화하면 아래의 그림과 같이 표현할 수 있다. Submodule Converter가 5개가 사용되면서 shift and add-3 알고리즘이 진행되고 있음을 diagram을 보면 파악이 가능하다. B0 ~ B3에 변환하고자 하는 Binary number를 input으로 각 digit을 입력하면 P9 ~ P0까지 BCD가 출력된다.

3. Verilog Implementation:

(1) Gate-Level Modeling of Converter Submodule :

// out1 $AD' + B'C + CD$

```
and_gate and_6(a(not_4_out), .b(in3), .out(and_6_out)); //  $AD'$ 
```

```
not_gate not_5(a(and_4_out), .out(not_5_out)); //  $B' + D$ 
```

```
and_gate and_7(a(not_5_out), .b(in1), .out(and_7_out)); //  $B'C + CD$ 
```

```
or_gate or_4(a(and_7_out), .b(and_6_out), .out(out1));
```

// out0 $AD' + BCD' + A'B'D$

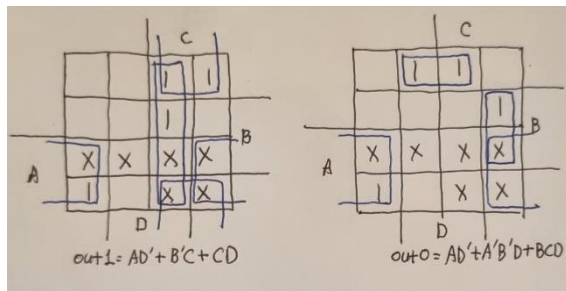
```
and_gate and_8(a(and_1_out), .b(not_4_out), .out(and_8_out)); //  $BCD'$ 
```

```
or_gate or_5(a(and_8_out), .b(and_6_out), .out(or_2_out)); //  $AD' + BCD'$ 
```

```
and_gate and_9(a(not_1_out), .b(not_2_out), .out(and_9_out)); //  $A'B'$ 
```

```
and_gate and_10(a(and_9_out), .b(in0), .out(and_10_out)); //  $A'B'D$ 
```

```
or_gate or_6(a(or_2_out), .b(and_10_out), .out(out0));
```



out3, out2의 Boolean expression(SOP)가 $A+BD+BC$, $AD+BC'D'$ 로 주어졌으므로 out1, out2의 SOP를 따로 구해서 적용했다. 둘의 Boolean expression을 K-Map을 통해 구하는 과정은 왼쪽의 그림과 같다. Wire로 and_out이 10개, or_out이 2개, not_out이 A', B', C', D'를 구하는 용도 외에 1개 더 선언되었기 때문에 out1을 구할 때 not을

이용했고 out0를 구할 때 남은 and_out의 개수에 맞춰 gates를 배치했다. 이에 대한 소스코드가 위와 같다.

(2) Structural Modeling of Binary_TO_BCD Converter :

```
assign BCD_code[0] = four_bit_binary[0];
```

```
submodule_dataflow_module mod_1(.in({0,0,0,four_bit_binary[3]}), .out({t[3],t[2],t[1],t[0]}));
```

```
submodule_dataflow_module mod_2(.in({0,0,0,t[3]}), .out({BCD_code[9],t[6],t[5],t[4]}));
```

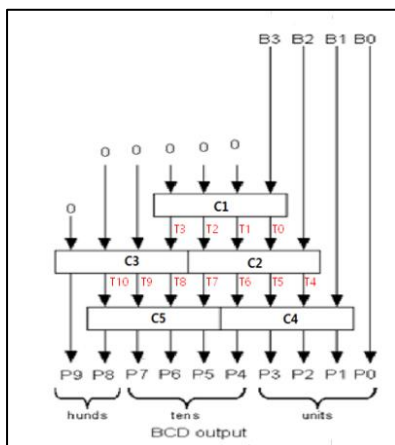
```
submodule_dataflow_module mod_3(.in({t[2],t[1],t[0],four_bit_binary[2]}), .out({t[7],t[6],t[5],t[4]}));
```

```
submodule_dataflow_module
```

```
mod_4(.in({t[10],t[9],t[8],t[7]}), .out({BCD_code[8],BCD_code[7],BCD_code[6],BCD_code[5]}));
```

```
submodule_dataflow_module
```

```
mod_5(.in({t[6],t[5],t[4],four_bit_binary[1]}), .out({BCD_code[4],BCD_code[3],BCD_code[2],BCD_code[1]}));
```



Theoretical Approach에서 설명한 왼쪽의 그림과 같이 Submodule 5개를 배치하여 structural modeling을 진행하였다. Binary number외의 입력값들은 모두 0으로 입력하고 wire로 선언된 [10 : 1] t를 C1 ~ C3까지의 outputs로 배치하여 모델링하였다. 최종적으로 [9 : 0] BCD_code를 도출하는 소스코드를 구현하였는데 이에 대한 코드는 위와 같다.

(3) Testbench of Modules :

```
//Submodule gatelevel
```

```
for (cnt = 0; cnt<16 ; cnt = cnt+1) begin
```

```
    #10 {IN3,IN2,IN1,IN0} = cnt;
```

```
end
```

// Binary-to-BCD converter

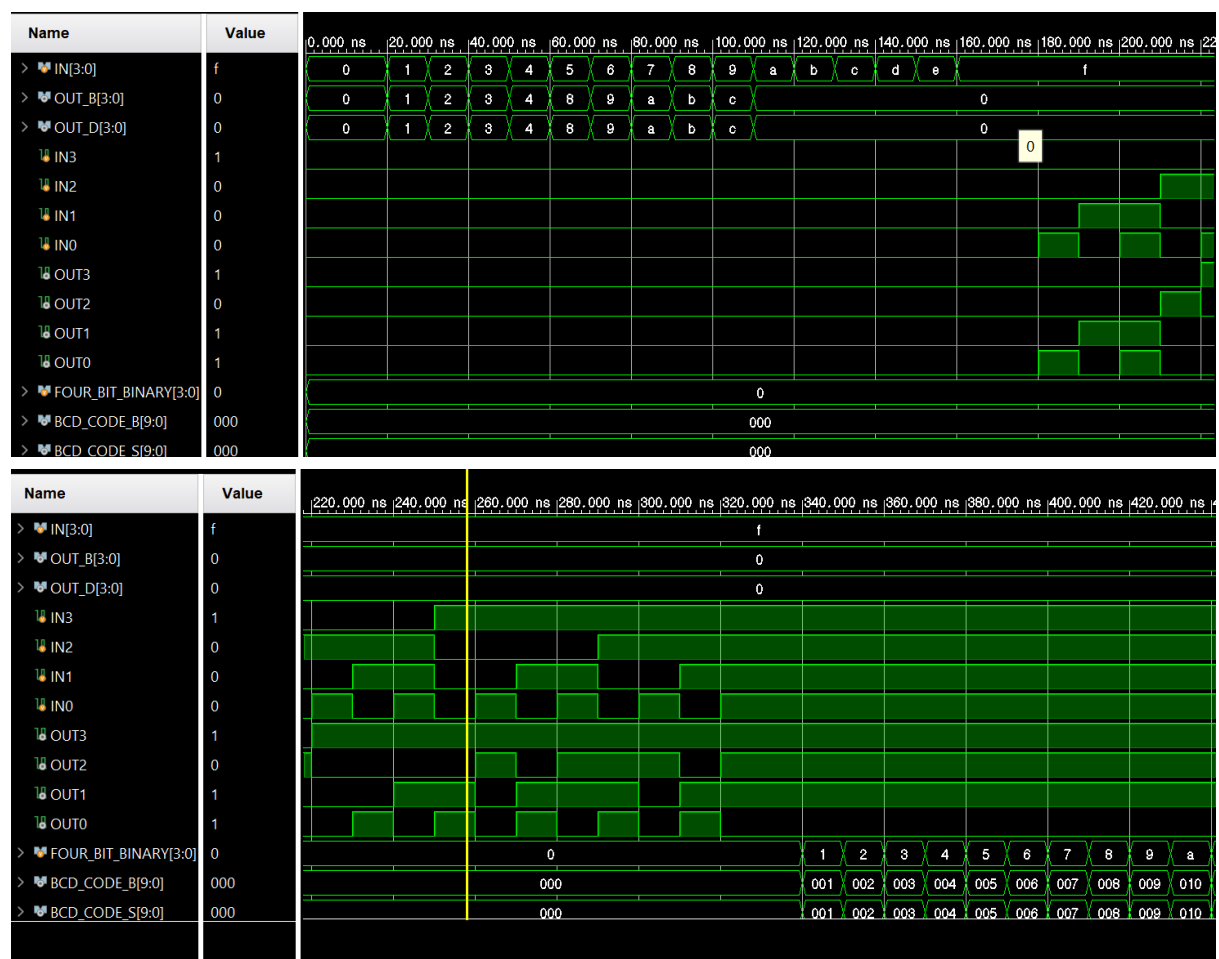
```
for (cnt = 0; cnt<16 ; cnt = cnt+1) begin
```

```
    #10 FOUR_BIT_BINARY <= cnt;
```

```
end
```

Submodule gatelevel의 경우 input이 IN3 ~ IN0까지 4개가 있다. 따라서 cnt<16으로 맞춰 #10에 따라 1씩 증가하도록 설정했다. 마찬가지로 Binary-to-BCD converter의 경우 input이 4-bit이기 때문에 cnt<16으로 맞춰 #10에 따라 1씩 증가하도록 설정했다.

(4) Simulation Results :



A3	A2	A1	A0	S3	S2	S1	S0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

Testbench에서 IN[3:0]에서 delay를 10ns만큼 주었기 때문에 총 160ns까지 지나고 바로 후부터 submodule의 test pattern이 적용되었음을 확인할 수 있다. 그리고 input이 1001에 해당되는 260ns까지 왼쪽의 Truth Table에 부합하는 OUT3 ~ OUT0가 도출되고 있음을 알 수 있다. 또한 Binary-to-BCD converter의 structural modeling도 이미 주어진 behavioral modeling과 정확히 일치하게 simulation이 진행되고 있음을 확인할 수 있다. 앞의 submodules에서 160ns + 160ns = 320ns가 지나고 test pattern이 시작되었고

이에 따라 10ns마다 behavioral modeling과 같은 값을 산출하고 있음이 simulation을 통해 알 수 있다.

5. Conclusion :

Shift and Add-3 알고리즘을 사용하는 Binary-to-BCD converter의 원리를 파악하여 submodule을 gate-level modeling으로 구축하고 structural modeling으로 submodule을 이용하여 완전한 converter의 소스코드를 구현하였다. Simulation을 진행한 결과 Gate-Level Modeling의 module의 inputs {IN3, IN2, IN1, IN0}가 0000부터 1001까지 정확히 Truth Table을 반영하고 있음을 확인하였다. 1010부터 1111까지는 XXXX의 값이 산출되는 것 또한 알 수 있다. 이 gate-level module을 submodule로 사용한 Binary-to-BCD converter의 simulation의 결과 또한 입력된 input과 같은 BCD가 output으로 산출되고 있음을 확인할 수 있었다. 이를 종합하여 고려해보면 디자인 한 submodule과 Binary-to BCD converter의 source code가 정확했다고 판단할 수 있다.